

# Advanced Software Engineering

## Comprehensive Self-Study Quiz Deck (Chapters 1–3)

"Tell me and I forget, teach me and I may remember, involve me and I learn." —  
Benjamin Franklin

 3 Core Chapters

 31 Conceptual Questions

 Instant Answer Feedback

 Self-Paced Learning

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

Software crisis, IEEE definition, core activities, Brooks's law, SDC, ISO 23010, Ethics & AI churn.

- **Questions:** Q01 – Q09 (9 Qs)
- **Target:** Core Mental Models

Start Chapter 1 →

## Chapter 2

### Process Models

Waterfall, V-Model, MVP, Agile Manifesto, Kanban WIP, Scrum, Tech Debt, XP, TDD & CI/CD.

- **Questions:** Q10 – Q23 (14 Qs)
- **Target:** Workflow & Agility

Start Chapter 2 →

#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete

# Chapter 1. Introduction to Software Engineering Foundations, Complexity, Professional Ethics & AI Impact

"The first false assumption: All programmers are created equal. The second: More programmers will make it faster." — *Fred Brooks*

 9 Questions (Q01 – Q09)

#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete

**Ch 1. Introduction • Question 01 Topic: Software Crisis & Complexity**

**A Why couldn't the 1968 Software Crisis be resolved simply by purchasing faster computer hardware or larger memory?**

**B A Computer hardware manufacturing and memory fabrication completely stagnated in the late 1960s, preventing computational speedups**

**C The crisis was fundamentally an intellectual and organizational challenge of system complexity, which faster hardware only amplified.**

C Programming languages of that era strictly lacked mathematical calculation primitives and compiler memory allocation capabilities.

D Early mainframe computers were physically incompatible with shared telecommunication networks and multi-terminal architectures.

👉 Click an option above to test your understanding

💡 **Correct Answer: B** — The crisis was fundamentally an intellectual and organizational failure in managing system complexity. Increasing hardware capacity allowed organizations to build systems of unprecedented scale, which human programmers using ad-hoc,

#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete

## Ch 1. Introduction • Question 02 Topic: IEEE Software Definition

According to the IEEE standard definition of software, which of the following is NOT considered a component of software?

A Executable computer programs and source code files.

B System database schemas and configuration files.

C CPU processor hardware and physical memory units.

D Software installation and deployment procedures.

👉 Click an option above to test your understanding

💡 **Correct Answer: C** — The IEEE standard defines software as computer programs, procedures, and possibly associated documentation and data. CPU hardware and physical memory are physical electronic devices (hardware) that execute software, rather than components of the software itself.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 1. Introduction • Question 03 Topic: Core Engineering Activities

Which of the following pairs correctly matches a specific software engineering action with its corresponding universal core activity?

A Conducting stakeholder interviews to draft user stories → Software Specification

B Writing automated unit tests to mock database responses → Software Design & Implementation

C Refactoring database schemas to improve query speed → Software Validation

D Swapping a third-party payment API for a new gateway → Software Specification

👉 Click an option above to test your understanding

💡 **Correct Answer: A** — Eliciting and modeling requirements through stakeholder interviews is a direct action in Software Specification. Writing unit tests is part of Software Validation (specifically verification). Refactoring database schemas is Software Evolution (preventive/perfective maintenance). Swapping APIs is Design & Implementation or Evolution.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck | Complete](#)

## Ch 1. Introduction • Question 04 Topic: Brooks's Law & Team Scaling

A project is 3 weeks behind schedule with 2 weeks remaining before release. The manager hires 4 junior programmers to speed up progress. What will happen according to Brooks's Law?

A The project will finish 1 week early.

B The project will be delayed further because senior engineers must spend time onboarding and mentoring new hires.

C The existing developers will code twice as fast.

D Communication complexity remains unchanged.

👉 Click an option above to test your understanding

💡 **Correct Answer: B** — Frederick Brooks demonstrated in The Mythical Man-Month that complex software development is not partitionable like manual labor. Adding people to a late project increases communication overhead quadratically according to  $n(n-1)/2$ , while senior engineers must stop productive coding to onboard newcomers.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 1. Introduction • Question 05 Topic: Separation of Concerns

An order-processing module directly handles HTTP requests, executes payment transactions, queries the SQL database, and generates HTML receipt emails. Which

fundamental design principle is most severely violated?

**A Separation of Concerns: Multiple distinct responsibilities are tightly tangled in a single module.**

**B YAGNI: Speculative future features are implemented before actual business requirements emerge.**

**C Brooks's Law: Adding developers to the order module increases communication complexity exponentially.**

**D Anticipation of Change: System configurations are hardcoded into compiled production binaries.**

👉 Click an option above to test your understanding

💡 **Correct Answer: A** — Separation of Concerns (and the Single Responsibility Principle)

dictates that a module should have only one reason to change and encapsulate a single

#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete

## Ch 1. Introduction • Question 06 Topic: ISO 25010 Quality Model

Which of the following matches a real-world software issue with its corresponding ISO 25010 quality characteristic?

A A database query taking 15 seconds to return results → Maintainability (Testability)

B A system crash occurring when a third-party API goes offline → Reliability (Fault Tolerance)

C Developers struggling to write unit tests due to tight coupling → Portability (Adaptability)

D An unencrypted session cookie allowing account takeover → Usability (Operability)

👉 Click an option above to test your understanding

💡 **Correct Answer: B** — A system's ability to cope with external service failures without crashing is the definition of Fault Tolerance (a sub-characteristic of Reliability). Slow query execution is Performance Efficiency (Time Behavior). Struggling to write unit tests is Maintainability (Testability). Unencrypted session cookies belong to Security (Confidentiality).

#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete

## Ch 1. Introduction • Question 07 Topic: ACM/IEEE Code of Ethics

Under the ACM/IEEE Code of Ethics, if an employer directs an engineer to implement an algorithm that falsifies safety compliance reports, what is the engineer's obligation?

A Comply, because the employer pays the engineer's salary.

B Refuse and escalate, because the Public Interest takes precedence over Employer loyalty.

C Implement the code but omit documentation.

D Outsource the code to an external vendor.

👉 Click an option above to test your understanding

💡 **Correct Answer: B** — Principle 1 of the ACM/IEEE Software Engineering Code of Ethics states that software engineers shall act consistently with the public interest, which takes absolute precedence over loyalty to an employer or client.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 1. Introduction • Question 08 Topic: Code Churn & AI Assistants

In empirical studies evaluating AI coding assistants (such as GitHub Copilot's analysis of 150M

lines of code), "Code Churn" emerged as a major warning sign. What does high Code

Churn indicate in an AI-assisted codebase?

A Code is rapidly rewritten, deleted, or patched shortly after commit, indicating brittle code

accepted without sufficient verification.

B Compilers and bundlers are aggressively removing unreachable dead code from

application binaries during automated deployment.

C Software engineering teams are switching programming languages frequently due to

automated polyglot syntax translation.

D Automated test cases are executing too quickly and depleting available CI/CD pipeline

virtual machine compute resources.

👉 Click an option above to test your understanding

💡 **Correct Answer: A** — Code churn measures the percentage of code that is modified, replaced, or deleted within two weeks of being committed. In AI coding environments, high

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

**Ch 1. Introduction • Question 09 Topic: GenAI Architectural Slop Risk**

An engineer prompts an AI to generate a complex payment calculation module, and then asks the same AI to write unit tests without providing a formal specification. All tests pass.

What is the primary risk?

**A Echo-chamber validation: The generated tests merely mirror the AI's internal flawed assumptions rather than actual business requirements.**

B Performance bottleneck: AI-generated test assertions take significantly longer to execute than human-written assertions.

C Compilation failure: Testing frameworks cannot parse automated mock datasets generated by large language models.

D Version lock-in: The test suite becomes tightly coupled to a single specific cloud runtime environment.

 Click an option above to test your understanding

 **Correct Answer: A** — When AI writes both the implementation and its own test cases without an independent specification contract, it falls into "echo-chamber testing"—

#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete

# Chapter 2. Software Development Processes & Methodologies

## Plan-Driven vs. Agile, Scrum, Kanban, Technical Debt & Extreme Programming

"Individuals and interactions over processes and tools; working software over comprehensive documentation." — *Agile Manifesto (2001)*

 14 Questions (Q10 – Q23)

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 2. Development Processes | Question 10 Topic: Waterfall Late Integration Risk

### What is the primary operational drawback of the traditional Waterfall model?

A It produces inadequate documentation for external auditing and compliance.

**B It makes accommodating changing requirements extremely difficult and costly once underway.**

C It eliminates the need for component and system testing during execution.

D It cannot be deployed across large-scale multi-site engineering organizations.

 Click an option above to test your understanding

 **Correct Answer: B** — The Waterfall model rigidly partitions activities into sequential stages with formal handoffs. Accommodating changing user needs requires costly backward iterations, contract renegotiation, and massive specification rework.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 2. Development Processes • Question 11 Topic: V-Model Verification & Validation

What is the primary engineering advantage of the V-Model over the classic Waterfall model?

A It produces working software increments in short two-week sprint iterations.

B It enforces test planning and acceptance criteria design concurrently with early specification phases.

C It eliminates the need for detailed architecture design and interface contracts.

D It allows customers to dynamically modify requirements at zero cost during implementation.

👉 Click an option above to test your understanding

💡 **Correct Answer: B** — The V-Model's definitive breakthrough is horizontal symmetry: test suites are authored concurrently with their corresponding specification phases (e.g., acceptance tests designed during requirements analysis), front-loading defect discovery.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 2. Development Processes • Question 12 Topic: Incremental vs. Iterative Process

In software process engineering, what is the fundamental conceptual difference between "Incremental" and "Iterative" development?

A Incremental focuses on automated testing; Iterative focuses on UI design.

B Incremental delivers finished functional slices stage-by-stage; Iterative refines an end-to-end working draft over repeated cycles.

C Incremental is managed by product owners; Iterative is managed exclusively by external regulators.

D Incremental follows waterfall rules; Iterative produces no documentation.

 Click an option above to test your understanding

 **Correct Answer: B** — Incremental builds software piece by piece by vertical functional slices; iterative starts with a rough draft of the entire system and progressively refines depth, fidelity, and polish through repeated cycles.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 2. Development Processes • Question 13 Topic: Kniberg's MVP Analogy (Skateboard)

In Henrik Kniberg's famous Minimum Viable Product (MVP) analogy (Skateboard to Car), why is delivering a standalone car wheel in the first release considered an anti-pattern?

A Because manufacturing an isolated wheel is significantly more expensive than building a skateboard.

B Because an isolated wheel provides zero end-to-end transportation value, preventing users from validating core problem assumptions.

C Because modern vehicle designs prohibit upgrading wheels into scooters.

D Because software engineering standards mandate that all early increments must be rectangular.

👉 Click an option above to test your understanding

💡 **Correct Answer: B** — An MVP must deliver standalone, end-to-end usable value that solves a real user problem. A single wheel cannot transport a person, leaving the user dissatisfied and generating zero empirical feedback on transportation needs.

#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete

Ch 2. Development Processes • Question 14 Topic: Agile Manifesto Living Software

The Agile Manifesto states: "Working software over comprehensive documentation."\*

What does this value primarily advocate in practice?

A Engineering teams are completely prohibited from writing architecture blueprints or API specifications.

B While documentation has value, delivering working, tested, and validated software is the primary measure of progress and customer value.

C Projects should be evaluated solely on executive PowerPoint presentations.

D Source code comments must be erased before deployment to production.

👉 Click an option above to test your understanding

💡 **Correct Answer: B** — The Agile Manifesto emphasizes that running, tested software delivering actual business value takes precedence over generating exhaustive paperwork. Documentation is authored when it serves an essential communicative purpose, but never at the expense of working software.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 2. Development Processes • Question 15 Topic: Agile Sustainable Development Pace

Agile Principle 8 states: **"Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely."\***

What is the primary engineering motivation?

**A To prevent code quality degradation, accumulated defects, and developer burnout caused by chronic overtime and crunch periods.**

B To mandate that developers submit at least 50 pull requests per day.

C To eliminate the need for software upgrades after initial system launch.

D To restrict development teams to working only on legacy systems.

 Click an option above to test your understanding

 **Correct Answer: A** — Chronic overtime causes severe cognitive fatigue, multiplying defect injection rates and accelerating developer burnout. A sustainable, predictable pace produces higher quality code and predictable delivery velocity over years.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 2. Development Processes • Question 16 Topic: Kanban WIP Limits & Flow Control

In the Kanban process framework, what is the primary operational purpose of enforcing

strict "Work In Progress" (WIP) limits on columns?

A To prevent developers from modifying automated unit test scripts.

B To expose bottlenecks, reduce multitasking context-switching waste, and maximize

delivery flow throughput.

C To mandate that every team member attends daily 15-minute standup meetings.

D To ensure all software increments are packaged into fixed 2-week sprint iterations.

👉 Click an option above to test your understanding

💡 **Correct Answer: B** — By restricting WIP limits, teams prevent hidden queues, eliminate context-switching waste, and immediately highlight process bottlenecks where cards pile up.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 2. Development Processes • Question 17 Topic: Daily Scrum Coordination Goals

In the Scrum framework, what is the primary operational objective of the \*Sprint

Retrospective\*\* held at the end of each sprint?

A To demonstrate working software increments to external business stakeholders.

B To inspect internal team collaboration, engineering practices, and tools, and identify

actionable process improvements for the next sprint.

C To assign individual performance ratings and conduct annual salary reviews.

D To rewrite the entire product backlog and discard unfinished user stories.

👉 Click an option above to test your understanding

💡 **Correct Answer: B** — While the Sprint Review focuses on inspecting the product with stakeholders, the Sprint Retrospective focuses inward on the team's processes, collaboration dynamics, and engineering practices to implement continuous self-improvement.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 2. Development Processes • Question 18 Topic: Technical Debt Metaphor

According to Ward Cunningham's Technical Debt metaphor, what represents the "compounding interest" paid by a software organization?

A The annual licensing fees paid for cloud hosting infrastructure and IDEs.

B The ongoing extra time, degraded velocity, and regression defects suffered during all future development.

C The bonus compensation paid to engineers who complete sprint tickets ahead of schedule.

D The legal costs of acquiring open-source third-party dependencies.

👉 Click an option above to test your understanding

💡 **Correct Answer: B** — The interest on technical debt is the extra friction and slowed velocity encountered every time engineers attempt to add new features or modify brittle, untested, poorly factored code.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

**Ch 2. Development Processes • Question 19 Topic: Martin Fowler's Flaccid Scrum**

**Martin Fowler coined the term "Flaccid Scrum" to describe which critical software engineering failure?**

**A Adopting Scrum management ceremonies (daily standups, sprints, story points) while completely neglecting technical engineering practices like TDD, refactoring, and CI**

**B Refusing to use Jira or commercial issue tracking software in favor of physical sticky notes.**

**C Allowing product owners to adjust backlog priorities between sprint planning sessions.**

**D Enforcing automated test execution on every Git commit in the continuous integration pipeline.**

👉 Click an option above to test your understanding

💡 **Correct Answer: A** — Flaccid Scrum occurs when organizations adopt agile project management rituals but ignore engineering craftsmanship. Without automated tests, refactoring, and clean architecture, code quickly becomes fragile and unmaintainable.

#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete

## Ch 2. Development Processes • Question 20 Topic: XP Pair Programming Roles

In Extreme Programming (XP), what is the primary role of the "Navigator" during a pair programming session?

A Typing out code syntax and executing local terminal commands.

B Thinking strategically, reviewing code in real time, considering edge cases, and looking at the broader architecture.

C Serving as the official sprint facilitator and managing Jira backlog ticket status.

D Negotiating customer contracts and approving annual engineering budgets.

👉 Click an option above to test your understanding

💡 **Correct Answer: B** — In pair programming, the Driver types the code while the Navigator thinks strategically, reviews code in real time, catches bugs, considers edge cases, and ensures the implementation aligns with overall architecture.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 2. Development Processes • Question 21 Topic: TDD Red-Green-Refactor Cycle

In Extreme Programming's Test-Driven Development (TDD), what is the specific objective of the **\*\*"Refactor"\*\*** step in the Red-Green-Refactor cycle?

A Adding new functional capabilities and expanding the module's public API contract.

B Improving the internal structure and readability of the code while ensuring all existing automated tests continue to pass.

C Removing unit tests that take longer than one second to execute in the local test suite.

D Rewriting the application from an object-oriented language to a functional programming language.

 Click an option above to test your understanding

 **Correct Answer: B** — Refactoring strictly means altering the internal structure of software to make it easier to understand and cheaper to modify without changing its observable external behavior, protected by passing unit tests.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 2. Development Processes • Question 22 Topic: Continuous Delivery vs. Deployment

What is the defining operational distinction between "Continuous Delivery" and "Continuous Deployment"?

A Continuous Delivery requires manual code compilation; Continuous Deployment automates compilation.

B Continuous Delivery stops at staging and requires human business approval to release; Continuous Deployment automatically deploys passing builds directly to live production.

C Continuous Delivery is used solely for mobile apps; Continuous Deployment is used solely for backend databases.

D Continuous Delivery eliminates unit testing; Continuous Deployment mandates pair programming.

👉 Click an option above to test your understanding

💡 **Correct Answer: B** — Continuous Delivery ensures that every build passing the automated pipeline is immediately deployable to production, but waits for a human business decision. Continuous Deployment automatically promotes every passing build

#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete

## Ch 2. Development Processes • Question 23 Topic: Automated CI/CD Quality Gates

In modern AI Specification-Driven development, what is the primary role of automated

CI/CD verification gates?

A To prevent human developers from reviewing artificial intelligence output.

B To enforce deterministic quality test compliance, and defect containment before AI-

generated code merges into production.

C To convert natural language prompts directly into cloud infrastructure invoices.

D To replace software specifications with unverified prompt histories.

👉 Click an option above to test your understanding

💡 **Correct Answer: B** — AI coding agents can generate code rapidly, but they may introduce subtle hallucinations or security flaws. Automated CI/CD gates provide deterministic verification barriers that prevent unverified code from reaching production.

#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete

## Chapter 3. Requirements Engineering Elicitation, Functional & Non-Functional Specifications, Modeling & Validation

"The hardest single part of building a software system is deciding precisely what to build." — *Fred Brooks (1987)*

 8 Questions (Q24 – Q31)

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 3. Requirements Engineering • Question 24 Topic: User vs. System Requirements

In requirements engineering, what is the critical operational distinction between **User Requirements** and **System Requirements**?

A User requirements are high-level stakeholder goals; system requirements are detailed functional contracts for developers.

B User requirements specify UI wireframes; system requirements specify backend database schemas.

C User requirements can never change; system requirements are refactored continuously during daily scrums.

D User requirements come from external legal auditors; system requirements are generated by compiler tools.

👉 Click an option above to test your understanding

💡 **Correct Answer: A** — User requirements communicate the overarching business capabilities to non-technical stakeholders, while system requirements bridge those desires into precise, contract-level engineering specifications.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

**Ch 3. Requirements Engineering • Question 25 Topic: Domain Requirements (UberEats)**

Consider the following specification for the UberEats platform: *“Due to municipal food hygiene regulations, perishable warm food delivery transit time shall not exceed 45 minutes, and containers must maintain a temperature above 60°C throughout transit.”*

What category of software requirement does this statement represent?

A Domain Requirement (a constraint imposed by the operational environment, industry regulations, or physical laws)

B Functional Requirement (a specification of an active software computation, user feature, or system service)

C Non-Functional Requirement (a general software quality attribute concerning performance, scalability, or uptime)

D User Requirement (a high-level, natural-language goal or business vision expressed by end consumers)

👉 Click an option above to test your understanding

💡 **Correct Answer: A** — While this requirement mentions a 45-minute timing constraint, it

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 3. Requirements Engineering • Question 26 Topic: Verifiable & Measurable NFRs

A client provides an imprecise non-functional goal: "The order checkout system must be

blazing fast and highly reliable."\* Which of the following correctly transforms this vague

goal into a \*\*verifiable, testable engineering metric\*\*?

A 99% of checkouts shall have response time  $\leq 500$  ms, and peak uptime shall be

$\geq 99.95\%$ .

B The checkout UI shall use sleek animations so customers perceive maximum speed.

C All checkout services shall use memory-safe code to guarantee bug-free execution.

D The cloud database shall allocate unlimited RAM whenever transaction load increases.

👉 Click an option above to test your understanding

💡 **Correct Answer: A** — A verifiable NFR must be quantifiable so that test engineers can objectively measure pass or fail. Specifying 95th- or 99th-percentile latency under 500 milliseconds and four-nines availability provides an unambiguous, enforceable engineering contract.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 3. Requirements Engineering • Question 27 Topic: Root Cause & The 5 Whys Heuristic

During a requirements elicitation interview, an executive insists: "Our clinical software

must include a blockchain ledger to record patient vitals." What is the most effective

engineering interview heuristic to apply?

A Apply the "5 Whys" to investigate the underlying data integrity and audit problem rather

than prematurely locking in the suggested technology.

B Immediately begin drafting smart contracts and relational database schemas for the

requested blockchain feature.

C Reject the executive's request outright because non-technical stakeholders are

prohibited from proposing system capabilities.

D Politely terminate the interview and switch exclusively to passive workplace observation.

👉 Click an option above to test your understanding

💡 **Correct Answer: A** — Stakeholders frequently suggest specific technical tools because they heard a buzzword, when their real business requirement is data integrity or regulatory

traceability. An engineer's job is to uncover the underlying problem, not prematurely

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

**Ch 3. Requirements Engineering • Question 28 Topic: Ethnography & Tacit Knowledge**  
**In requirements engineering, why is *\*\*Ethnography (workplace observation)\*\** uniquely vital when analyzing complex operational environments such as hospital emergency rooms or air traffic control?**  
**A It uncovers tacit knowledge—ingrained habits, physical workarounds, and unwritten shortcuts that users perform automatically but never mention during interviews**  
**B It automatically compiles natural language requirements directly into executable**

**acceptance test suites without human intervention.**

**C It eliminates the need for subsequent software architecture design, database modeling, or code review phases.**

**D It guarantees that the resulting software requirements will achieve 100% mathematical completeness on the initial development sprint.**

 Click an option above to test your understanding

 **Correct Answer: A** — People often cannot articulate everything they do because habits become second nature. By observing practitioners in their actual physical workspace,

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck | Complete](#)

## Ch 3. Requirements Engineering • Question 29 Topic: Use Case Diagrams vs. Descriptions

A software engineering team creates a UML Use Case Diagram showing an actor connected to the "Withdraw Cash" use case. Why must engineers author a detailed textual **\*Use Case Description\*** in addition to the diagram?

A Diagrams only show high-level scope; descriptions define sequential flows, preconditions, postconditions, and exception handling.

B UML diagrams cannot be rendered by web browsers without accompanying markdown text.

C Compilers require use case descriptions to allocate heap memory for actor threads.

D Descriptions convert non-functional requirements into automated GUI wireframes.

👉 Click an option above to test your understanding

💡 **Correct Answer: A** — A use case diagram is just a visual table of contents; it shows who interacts with what feature. But engineers cannot write code or tests from an ellipse alone. They need the textual use case description to know the exact preconditions, the sequential normal flow, and how to handle exception flows when things go wrong.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 3. Requirements Engineering • Question 30 Topic: Exponential Cost of Late Defects

Why is fixing a requirements error after software delivery significantly more expensive than fixing an error during early development?

A Requirements documents cannot be legally modified once signed by clients.

B A late fix requires redesigning, recoding, retesting, and redeploying cascading components that were built on the flawed premise.

C Compilers automatically lock code repositories against changes after the first production release.

D Automated unit tests lose their validity after code has been deployed to cloud environments.

👉 Click an option above to test your understanding

💡 **Correct Answer: B** — When a requirement defect escapes into production, all the downstream work built on that false premise—the architectural design, database schemas, API contracts, frontend views, and automated tests—must be ripped out and rebuilt. That cascading rework is what causes the 100x cost explosion.

[#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete](#)

## Ch 3. Requirements Engineering • Question 31 Topic: LLM Hallucinations in Requirements

When software engineering teams use Large Language Models (LLMs) to generate requirements specifications from stakeholder interview transcripts, what is the primary operational risk requiring human-in-the-loop verification?

A The LLM may hallucinate plausible-sounding but fictitious business logic, omitted edge-case constraints, and non-existent external API integrations.

B The LLM cannot output text formatted in Markdown bullet points or standard user story

Given-When-Then acceptance criteria.

C The LLM will consume excessive server memory and cause database deadlock errors across production microservice clusters.

D The LLM strictly enforces waterfall development practices and refuses to generate requirements for iterative agile sprint cycles.

👉 Click an option above to test your understanding

💡 **Correct Answer: A** — LLMs are fluent text predictors, not domain experts. They

frequently invent plausible-sounding business rules or overlook subtle safety constraints

#02ASE Self-Study Quiz Deck ▾ | Master Table of Contents#03Ch 1. Introduction | Overview ▾#04Ch 1. Introduction | Q01 of 31 ▾#05Ch 1. Introduction | Q02 of 31 ▾#06Ch 1. Introduction | Q03 of 31 ▾#07Ch 1. Introduction | Q04 of 31 ▾#08Ch 1. Introduction | Q05 of 31 ▾#09Ch 1. Introduction | Q06 of 31 ▾#10Ch 1. Introduction | Q07 of 31 ▾#11Ch 1. Introduction | Q08 of 31 ▾#12Ch 1. Introduction | Q09 of 31 ▾#13Ch 2. Development Processes | Overview ▾#14Ch 2. Development Processes | Q10 of 31 ▾#15Ch 2. Development Processes | Q11 of 31 ▾#16Ch 2. Development Processes | Q12 of 31 ▾#17Ch 2. Development Processes | Q13 of 31 ▾#18Ch 2. Development Processes | Q14 of 31 ▾#19Ch 2. Development Processes | Q15 of 31 ▾#20Ch 2. Development Processes | Q16 of 31 ▾#21Ch 2. Development Processes | Q17 of 31 ▾#22Ch 2. Development Processes | Q18 of 31 ▾#23Ch 2. Development Processes | Q19 of 31 ▾#24Ch 2. Development Processes | Q20 of 31 ▾#25Ch 2. Development Processes | Q21 of 31 ▾#26Ch 2. Development Processes | Q22 of 31 ▾#27Ch 2. Development Processes | Q23 of 31 ▾#28Ch 3. Requirements Engineering | Overview ▾#29Ch 3. Requirements Engineering | Q24 of 31 ▾#30Ch 3. Requirements Engineering | Q25 of 31 ▾#31Ch 3. Requirements Engineering | Q26 of 31 ▾#32Ch 3. Requirements Engineering | Q27 of 31 ▾#33Ch 3. Requirements Engineering | Q28 of 31 ▾#34Ch 3. Requirements Engineering | Q29 of 31 ▾#35Ch 3. Requirements Engineering | Q30 of 31 ▾#36Ch 3. Requirements Engineering | Q31 of 31 ▾#37ASE Self-Study Quiz Deck ▾ | Complete

Revisit Software Crisis, ISO 25010 & Ethics concepts.

Jump to Q01 →

## Review Ch 2

Revisit Agile, Scrum, Kanban & TDD cycles.

Jump to Q10 →

## Review Ch 3

Revisit Elicitation, NFR metrics & Use Cases.